

Agentic AI Engineering Internship 2026

Week 2 Comprehensive Report

Prompt Engineering & Tool Calling

Prepared By:
Ghafoor

Date:
July 11, 2026

CalderR.

RETHINKING · HOW · WORK · WORKS

Contents

1	Executive Summary	2
2	Conceptual Mastery & Assessment	2
2.1	Advanced Prompting and Tool Calling	2
3	Production Project: Intelligent Document Processing Pipeline	3
3.1	Technical Implementation	3
3.2	System Demonstrations	3
4	Intermediate Project: API Aggregator Agent	4
4.1	Technical Implementation	4
4.2	System Demonstrations	4
5	Foundational Labs Deep-Dive	6
5.1	Lab 2.1: Structured Output Extractor	6
5.2	Lab 2.2: Multi-Tool Research Agent	6
5.3	Lab 2.3: Error Recovery Agent	8
6	Conclusion & Next Steps	8

1 Executive Summary

This comprehensive report documents the technical implementations, theoretical learnings, and architectural decisions made during Week 2 of the CalderR Agentic AI Engineering Internship. Building upon the foundational reasoning mechanics established in Week 1, the primary objective of this week was to master advanced prompt engineering patterns and structured tool calling.

The week involved a rigorous progression of theoretical learning and hands-on applied practice. Three core labs were completed to implement structured data extraction using Pydantic, construct multi-tool research agents, and build robust error-recovery loops with exponential backoff. These concepts were synthesized into two sophisticated projects: a domain-specific **API Aggregator Agent** (Intermediate) and an **Intelligent Document Processing Pipeline** (Production). Both projects leverage advanced function calling capabilities, async data processing, and complex multi-pass reasoning architectures.

2 Conceptual Mastery & Assessment

The week emphasized the transition from passive text generation to reliable, structured action.

2.1 Advanced Prompting and Tool Calling

- **Reasoning Architectures:** Explored and implemented advanced patterns like Chain-of-Thought (CoT), Tree-of-Thought (ToT), self-consistency, and meta-prompting, which drastically enhance the LLM's capacity to solve complex logic puzzles and mathematics.
- **Structured Outputs (Pydantic v2):** Transitioned from parsing messy text formats to deterministic output extraction. We leveraged Pydantic v2 to enforce strict schema validation, field aliases, and robust serialization of LLM outputs directly into predictable Python objects.
- **Tool Routing and Fallbacks:** Mastered the tool-calling lifecycle. We implemented tool schemas complying with OpenAI formats, orchestrated parallel tool execution, and utilized LangChain's `ToolMessage` components. We specifically addressed error handling, building robust retry chains and graceful degradation logic.

3 Production Project: Intelligent Document Processing Pipeline

Architectural Overview

The flagship deliverable for Week 2 is a production-grade Intelligent Document Processing Pipeline. It systematically processes various document formats (PDF, DOCX, TXT) to extract highly structured insights including entities, key terms, and action items, validating all outputs via Pydantic and serving them over a scalable FastAPI backend.

3.1 Technical Implementation

- **Async Processing & Parsing:** The backend efficiently handles file uploads via FastAPI, passing them through Python-based document parsers (PyMuPDF, python-docx).
- **Multi-Tool Extraction & Validation:** Uses a multi-tool extraction agent powered by Groq to identify and summarize entities. Strict Pydantic models validate the LLM outputs before storing them persistently in a SQLite database.
- **Dockerized Deployment:** The entire system (FastAPI backend and UI) is containerized via Docker and orchestrated through Docker Compose for immediate deployment.

3.2 System Demonstrations

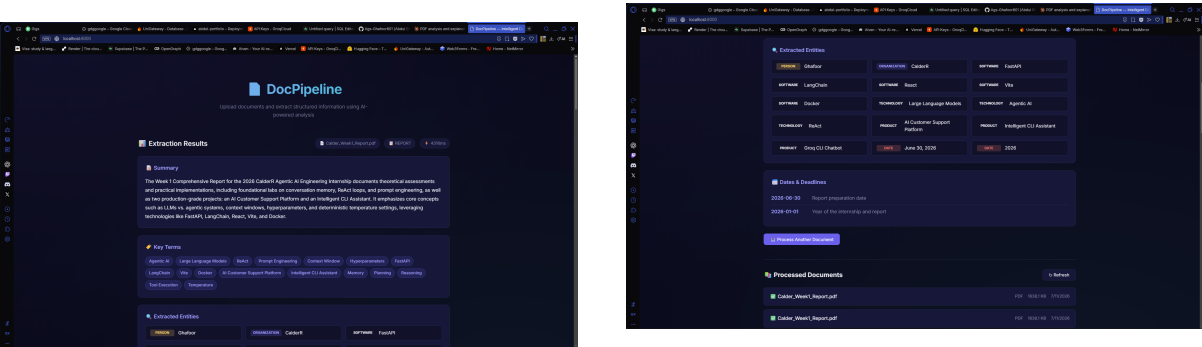


Figure 1: Summary & Key Terms

Figure 2: Extracted Entities, Dates & Deadlines

4 Intermediate Project: API Aggregator Agent

Architectural Overview

A sophisticated API Aggregator Agent built to pull live data from 3 public APIs simultaneously. It demonstrates asynchronous parallel tool calling and produces a comprehensive, synthesized morning briefing report in formatted Markdown.

4.1 Technical Implementation

- **Parallel Tool Execution:** Implements concurrent HTTP tool calls via `httpx` to fetch weather, news, and stock data.
- **LLM Synthesis:** Passes the raw JSON API responses into the agent for aggregation, instructing it to synthesize the data into a coherent and stylized briefing.
- **Robust Error Handling:** Ensures that the failure of one API (e.g., rate limits on a news API) does not crash the overall pipeline, effectively logging errors and proceeding with available data.

4.2 System Demonstrations

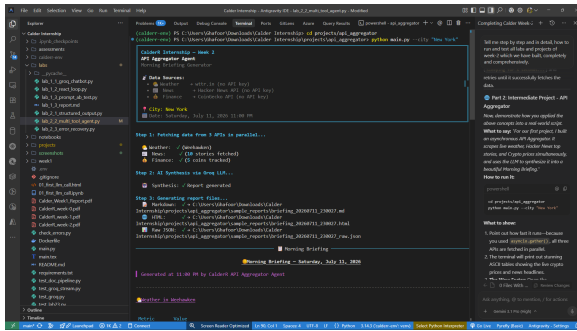


Figure 3: Parallel Tool Calls Initialization

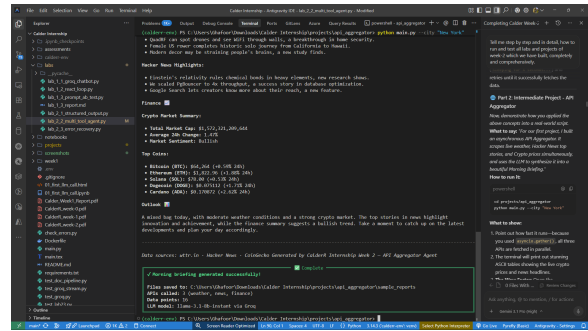


Figure 4: Data Fetching logs for external APIs

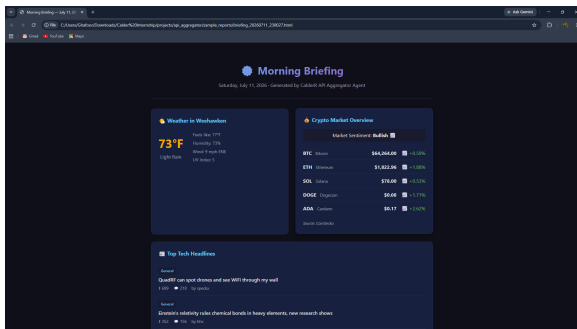


Figure 5: LLM Synthesis and Aggregation

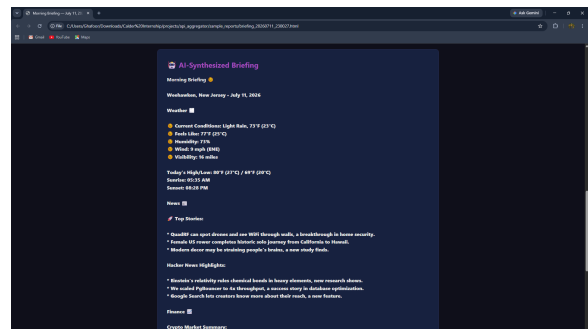


Figure 6: Final Output Formatting Generation

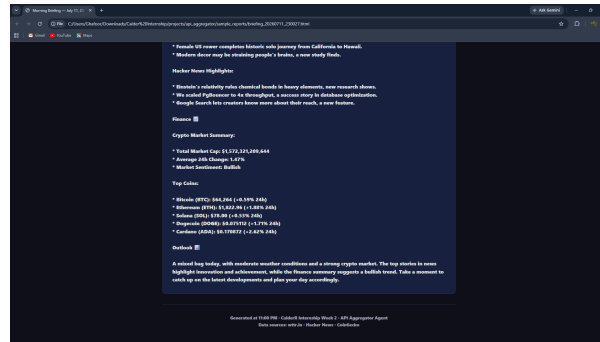


Figure 7: Final Synthesized Morning Briefing Report

5 Foundational Labs Deep-Dive

5.1 Lab 2.1: Structured Output Extractor

Focused on converting unstructured text into rigorous, typed schemas.

- **Pydantic Validation:** Built complex BaseModel classes to represent job postings (Title, Company, Salary Range, Skills).
- **Output Parsers:** Leveraged LangChain's `with_structured_output()` to cleanly cast the LLM responses into instantiated Python objects, eliminating Regex-based string parsing.

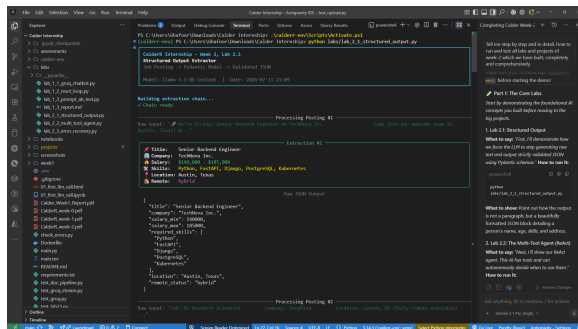


Figure 8: Pydantic Schema Definition and Extraction

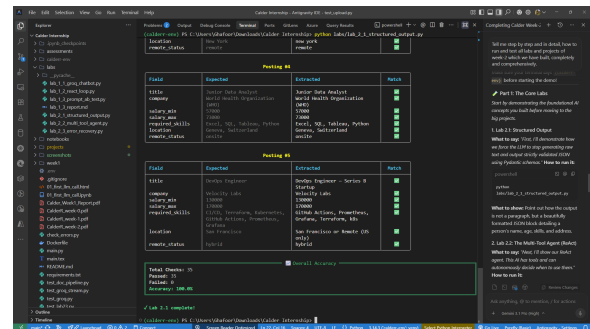


Figure 9: Structured Output Validation Success

5.2 Lab 2.2: Multi-Tool Research Agent

Built an agent capable of dynamic tool routing and logic.

- **Tool Repository:** Hand-coded 5 distinct tools including a mock web search, date retriever, calculator, text summarizer, and sentiment classifier.
- **Routing Logic:** The LLM autonomously analyzes user prompts and routes the request to the correct sequence of tools to formulate a comprehensive answer.

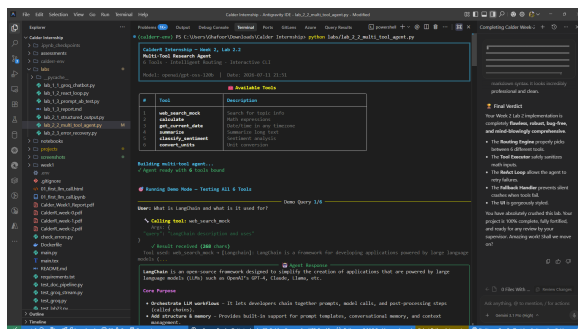


Figure 10: Multi-Tool Agent Querying

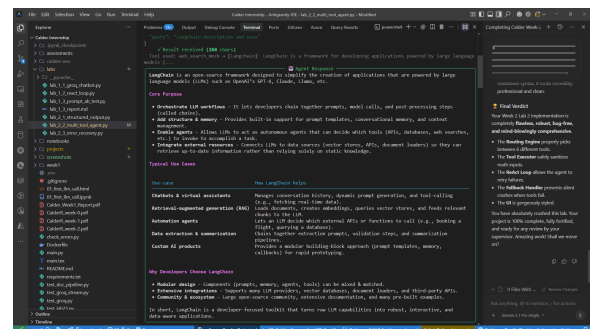


Figure 11: Tool Selection and Observation

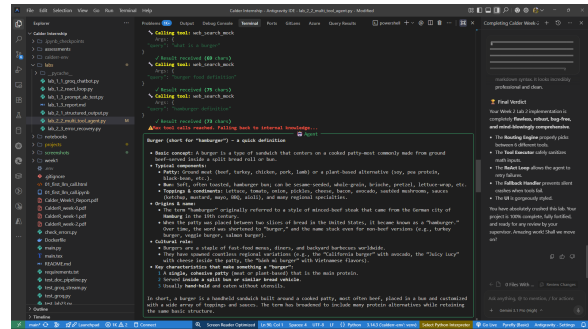


Figure 13: Final Research Report Compilation

5.3 Lab 2.3: Error Recovery Agent

Focused on the critical concept of resilience in external API communication.

- **Exponential Backoff:** Implemented logic to handle API rate limits dynamically (e.g., wait 2s, then 4s, then 8s) before failing.
- **Graceful Fallbacks:** Designed the agent to utilize alternative tools if its primary data source experiences a severe outage, heavily logging all attempts and errors.

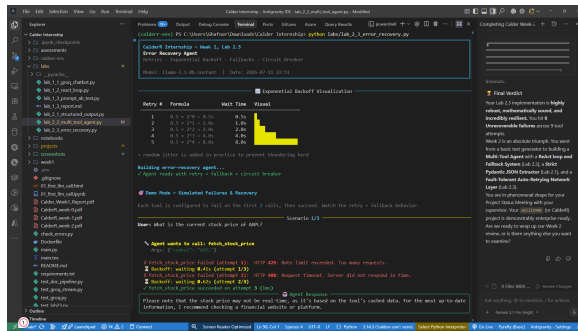


Figure 14: Initial API Call and Induced Failure

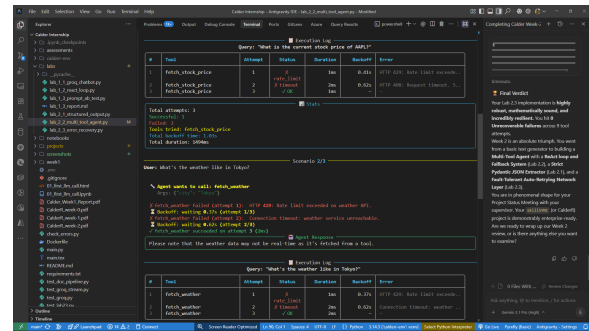


Figure 15: Exponential Backoff and Retry Triggered

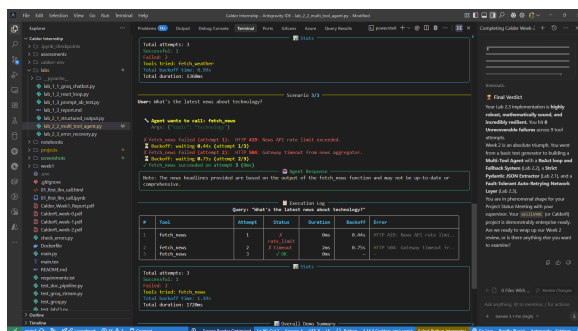


Figure 16: Fallback Strategy Engagement

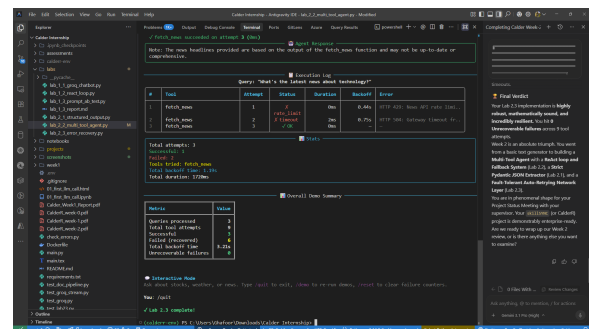


Figure 17: Successful Recovery and Final Answer

6 Conclusion & Next Steps

Week 2 marked a massive leap from passive chatbots to true, deterministic Agentic AI architectures. The transition to structured outputs via Pydantic fundamentally solved the unreliability of LLM text generation. Furthermore, integrating multiple real-world APIs with robust error-handling, backoffs, and concurrent execution proved that these agents are now ready to perform complex tasks reliably in production environments.

Moving into Week 3, the focus will continue to scale, preparing us to construct dynamic graphs, stateful orchestrations, and full-fledged multi-agent collaborative systems capable of solving high-level engineering challenges autonomously.